

A Mathematical Lecture on a Heterogeneous GraphSAGE Recommender

PhD-Level Lecture Notes in Applied Mathematics, AI Engineering, and Data Science

May 20, 2026

Contents

1	Overview	2
2	Configuration as a Mathematical Specification	3
2.1	Graph-Build Configuration	3
2.2	Node Feature Configuration	3
2.3	Model Configuration	4
3	Graph Construction	4
3.1	Raw View	4
3.2	User Nodes	5
3.3	Bundle Nodes	5
3.4	Edges and Temporal Splits	5
4	Feature Encoding	5
4.1	Numerical Standardization	6
4.2	Categorical Encoding	6
5	Tabular Encoders	6
6	Heterogeneous GraphSAGE Model	7

7	Link-Prediction Scoring	7
8	Training Objective	7
9	Evaluation Metrics	8
9.1	ROC AUC	8
9.2	Average Precision	8
10	Inference for Known Users	8
11	Cold-Start Recommendations	9
11.1	Cohort Matching	9
11.2	Cohort Popularity Score	9
12	Bulk Recommendation Export	9
13	Computational Complexity	10
14	Strengths of the Design	10
15	Limitations and Modeling Caveats	10
16	Compact Mathematical Summary	11
17	Conclusion	11

1 Overview

These notes explain, from first principles, the GraphSAGE recommendation system implemented in the supplied Python code. The system builds a heterogeneous graph from telecom subscription events, trains a neural link-prediction model, and exports ranked bundle recommendations for users.

The central task is the following: given historical interactions between users and bundles, estimate which unseen bundle a user is likely to subscribe to next. Mathematically, this is a bipartite link-prediction problem. The model observes edges of the form

$$(u, b) \in \mathcal{U} \times \mathcal{B},$$

where u is a user and b is a bundle. It learns an embedding vector for each user and bundle, then scores candidate pairs by an inner product.

The implementation combines four major ideas:

1. **relational data engineering:** raw parquet files are aggregated into user nodes, bundle nodes, and train/validation/test edges;
2. **heterogeneous graph learning:** users and bundles are different node types connected by typed edges;
3. **GraphSAGE message passing:** node representations are updated by sampling and aggregating neighbors;
4. **link prediction:** the model learns to distinguish observed user–bundle edges from sampled negative edges.

The code is therefore a full machine-learning pipeline rather than a single model class. It includes configuration objects, feature encoders, graph construction, training, evaluation, inference, cold-start fallbacks, and bulk recommendation export.

2 Configuration as a Mathematical Specification

The configuration defines three layers of the system.

2.1 Graph-Build Configuration

The graph builder reads parquet files from an input directory, writes graph artifacts into a model directory, and splits time by two integer dates:

$$t_{\text{train}} = 20251010, \quad t_{\text{val}} = 20251020.$$

Events with dates up to t_{train} define the training graph. Events after that date and up to t_{val} define validation labels. Later events define test labels.

This is a temporal split, not a random split. That choice matters: recommendation systems should usually be evaluated by predicting future behavior from past behavior.

2.2 Node Feature Configuration

The user node has numerical features such as raw event count, active days, total revenue, average price, and counts of subscriptions by service type. It also has categorical features such as service class, channel, payment mode, city, brand, and device capability.

The bundle node has numerical features such as price, configured data volume, configured voice minutes, configured SMS volume, validity, distinct users, and revenue. Its categorical feature is the bundle type.

For a user u , write its numerical vector as

$$x_u^{\text{num}} \in \mathbb{R}^{d_u},$$

and its categorical vector as

$$x_u^{\text{cat}} \in \mathbb{N}^{c_u}.$$

Similarly, a bundle b has vectors x_b^{num} and x_b^{cat} .

2.3 Model Configuration

The GraphSAGE configuration sets the hidden dimension, categorical embedding dimension, number of message-passing layers, dropout, batch sizes, negative-sampling ratios, learning rate, weight decay, evaluation limits, and random seed.

The neighbor-sampling schedule is

$$\{15, 10\}$$

for both the forward edge type and the reverse edge type. Thus, for a two-layer model, the sampler first takes up to 15 neighbors at one hop and then up to 10 neighbors at the next hop. This bounds the memory cost of training on large graphs.

3 Graph Construction

The graph builder converts raw event rows into a heterogeneous graph

$$G = (\mathcal{V}, \mathcal{E}), \quad \mathcal{V} = \mathcal{U} \cup \mathcal{B}.$$

There are two node types:

$$\text{user}, \quad \text{bundle},$$

and two edge types:

$$(\text{user}, \text{subscribed_to}, \text{bundle}), \quad (\text{bundle}, \text{rev_subscribed_by}, \text{user}).$$

The reverse edge type is not new information; it is a computational device that lets messages flow from bundles back to users in a heterogeneous graph neural network.

3.1 Raw View

The graph builder first creates a normalized DuckDB view called **raw**. Each field is cast to a controlled type: dates become integers, identifiers become strings, prices and volumes become doubles, and rows with missing user IDs, bundle IDs, or dates are removed.

This is an important data-engineering step. Graph neural networks are sensitive to indexing consistency: every user and bundle must have a stable integer ID before tensors can be built.

3.2 User Nodes

For each user u , the code aggregates all training-period rows with that user’s MSISDN. The resulting user features include

$$\begin{aligned}\text{raw_events}(u) &= \#\{\text{training rows for } u\}, \\ \text{active_days}(u) &= \#\{\text{distinct dates for } u\}, \\ \text{distinct_bundles}(u) &= \#\{\text{distinct bundles used by } u\}, \\ \text{total_revenue}(u) &= \sum_{i:u_i=u} \text{total_rev}_i.\end{aligned}$$

Categorical fields are taken from the most recent observed training row using an **ARG_MAX** operation.

Finally, the code assigns

$$\text{user_idx}(u) \in \{0, 1, \dots, |\mathcal{U}| - 1\}$$

by sorting users by MSISDN.

3.3 Bundle Nodes

For each bundle b , the code aggregates training-period rows into one node. It stores the latest bundle name, type, and price, and it averages configured volumes and validity. It also records popularity quantities such as distinct users, total subscriptions, and total revenue.

The bundle index is similarly assigned as

$$\text{bundle_idx}(b) \in \{0, 1, \dots, |\mathcal{B}| - 1\}.$$

3.4 Edges and Temporal Splits

Edges are aggregated by user–bundle pair. For a split S , the edge table contains

$$(u, b, \text{raw_events}, \text{active_days}, \text{subscriptions}, \text{revenue}, \text{price}).$$

Only users and bundles present in the training graph are retained in validation and test edges. This means the supervised evaluation is transductive: the model predicts future links among known node sets.

4 Feature Encoding

The feature module transforms tabular node features into numerical tensors.

4.1 Numerical Standardization

For each numerical column j , the training mean and standard deviation are computed:

$$\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}, \quad s_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \mu_j)^2}.$$

Missing and infinite values are handled by replacing infinities with missing values, filling missing values with μ_j , and applying

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{s_j}.$$

If a standard deviation is zero, it is replaced by 1 to avoid division by zero.

4.2 Categorical Encoding

Each categorical column is mapped to integer IDs. If a column has observed values

$$\{a_0, a_1, \dots, a_{m-1}\},$$

then the encoder stores a dictionary $a_i \mapsto i$ and reserves index m for unknown values. The cardinality passed to the neural embedding layer is therefore $m + 1$.

This is why the inference code must load the saved categorical maps: production recommendations must use the same category-to-index mapping learned during training.

5 Tabular Encoders

Before message passing begins, users and bundles are embedded by a shared architectural pattern called **TabEncoder**.

For categorical feature j with cardinality C_j , the code creates an embedding table

$$E_j \in \mathbb{R}^{C_j \times e_j},$$

where

$$e_j = \min\{\text{cat_emb_dim}, \max(4, \lfloor \sqrt{C_j} \rfloor)\}.$$

The encoder concatenates standardized numerical features and categorical embeddings:

$$h_v^{(0)} = \phi \left(W \left[x_v^{\text{num}} \parallel E_1(x_{v1}^{\text{cat}}) \parallel \dots \parallel E_c(x_{vc}^{\text{cat}}) \right] + a \right),$$

where ϕ consists of a linear layer, ReLU activation, layer normalization, and dropout.

Separate encoders are used for users and bundles because the two node types have different feature schemas.

6 Heterogeneous GraphSAGE Model

The model uses **HeteroConv** with two **SAGEConv** relations. At each layer ℓ , user embeddings receive messages from neighboring bundles through the reverse relation, and bundle embeddings receive messages from neighboring users through the forward relation.

In abstract form, a GraphSAGE update can be written as

$$h_v^{(\ell+1)} = \psi \left(W_1 h_v^{(\ell)} + W_2 \cdot \text{AGG}\{h_w^{(\ell)} : w \in \mathcal{N}(v)\} \right),$$

where AGG is the aggregation operation implemented by the convolution layer and ψ includes normalization, ReLU, and dropout.

Because the graph is heterogeneous, the model keeps a dictionary of embeddings:

$$\{\text{user} : H_U^{(\ell)}, \text{bundle} : H_B^{(\ell)}\}.$$

After each convolution, user and bundle embeddings are normalized separately.

Remark 1. *The reverse edge relation is essential for symmetric information flow. Without it, one node type could receive updates while the other might not receive the intended messages under the typed-edge convention.*

7 Link-Prediction Scoring

The model scores a candidate pair (u, b) using the dot product between the final user embedding and final bundle embedding:

$$z_u, z_b \in \mathbb{R}^d, \quad \text{score}(u, b) = z_u^\top z_b.$$

This scalar is a logit. The corresponding probability-like score is

$$\hat{p}_{ub} = \sigma(z_u^\top z_b) = \frac{1}{1 + e^{-z_u^\top z_b}}.$$

The recommendation ranking is based on these scores. Since the sigmoid function is strictly increasing, ranking by logits and ranking by sigmoid probabilities are equivalent.

8 Training Objective

Training uses observed edges as positive examples. Negative examples are sampled from user–bundle pairs not presented as positive labels in a mini-batch. If $y_{ub} = 1$ for observed edges and $y_{ub} = 0$ for sampled negatives, the loss is binary cross-entropy with logits:

$$\mathcal{L} = -\frac{1}{N} \sum_{(u,b)} [y_{ub} \log \sigma(s_{ub}) + (1 - y_{ub}) \log(1 - \sigma(s_{ub}))],$$

where

$$s_{ub} = z_u^\top z_b.$$

The implementation trains with AdamW, gradient clipping, dropout, optional mixed precision on CUDA, and neighbor-sampled mini-batches. The best checkpoint is selected by validation average precision.

9 Evaluation Metrics

The code computes two threshold-free ranking metrics on sampled validation and test edges.

9.1 ROC AUC

The ROC AUC estimates the probability that a randomly chosen positive edge receives a higher score than a randomly chosen negative edge:

$$\text{AUC} = \Pr(s^+ > s^-).$$

It measures global pairwise separability.

9.2 Average Precision

Average precision summarizes the precision–recall curve and is often more informative than AUC for sparse recommendation graphs, where positive edges are rare compared with all possible user–bundle pairs.

The trainer stores the best model by validation AP:

$$\text{best checkpoint} = \arg \max_{\text{epoch}} \text{AP}_{\text{val}}.$$

10 Inference for Known Users

For a known user, the recommender scores that user against all candidate bundles. For a set of users U' and all bundles B , it forms the Cartesian product

$$U' \times B$$

and evaluates the trained model in batches.

If `exclude_seen` is enabled, any bundle already present in the user’s training history is assigned score $-\infty$ before ranking. The top- k recommendation set is then

$$\text{TopK}(u) = \arg \text{topk}_{b \in B} \sigma(z_u^\top z_b).$$

The output row contains rank, MSISDN, user index, bundle index, bundle ID, bundle name, bundle type, price, score, recommendation type, matched features, and cohort size.

11 Cold-Start Recommendations

The model cannot directly embed a user whose MSISDN was not present in the training graph, because that user has no node index and no graph neighborhood. The code therefore implements a non-neural cold-start fallback.

11.1 Cohort Matching

Given a partial profile, the recommender tries to find a cohort of existing users matching selected categorical attributes. The priority order is

service class, payment mode, device capability, brand, city, channel.

If the exact matching cohort is too small, the least-prioritized feature is dropped and the search repeats. This is a form of progressive relaxation.

11.2 Cohort Popularity Score

For the resulting cohort, bundles are scored by a weighted combination of normalized popularity and revenue features:

$$S(b) = 0.35 M(\text{cohort_users}_b) + 0.25 M(\text{cohort_subscriptions}_b) \\ + 0.20 M(\text{cohort_revenue}_b) + 0.10 M(\text{distinct_users}_b) + 0.10 M(\text{total_subscriptions}_b),$$

where M is min-max normalization.

If no suitable cohort exists, the system falls back to global popularity:

$$S_{\text{global}}(b) = 0.50 M(\text{distinct_users}_b) + 0.30 M(\text{total_subscriptions}_b) + 0.20 M(\text{total_revenue}_b).$$

12 Bulk Recommendation Export

The export method computes full-graph embeddings once, then scores users in chunks. If

$$Z_U \in \mathbb{R}^{n_U \times d}, \quad Z_B \in \mathbb{R}^{n_B \times d},$$

then all logits for a chunk of users are computed by a matrix multiplication:

$$L = Z_{U,\text{chunk}} Z_B^\top.$$

This is much faster than repeatedly invoking the neighbor loader for every candidate pair. The method then applies `topk` to each row, converts logits to sigmoid scores, and writes recommendations to CSV.

13 Computational Complexity

Let n_U be the number of users, n_B the number of bundles, d the hidden dimension, and k the number of recommendations requested.

Graph construction is dominated by grouped SQL aggregations over raw event rows. Training cost depends on the number of sampled mini-batches and sampled neighborhoods rather than the full graph size. Inference for all users by dense matrix multiplication costs approximately

$$O(n_U n_B d)$$

for scoring, followed by top- k selection per user.

The neighbor-sampling design is what makes training scalable, while the full-embedding export path is what makes large-scale batch inference efficient.

14 Strengths of the Design

This implementation has several strong engineering and mathematical properties:

1. **Temporal evaluation:** validation and test edges occur after the training window.
2. **Typed graph structure:** user and bundle features are represented with separate schemas and encoders.
3. **Scalable training:** neighbor sampling avoids loading full computation neighborhoods for every batch.
4. **Interpretable fallbacks:** cold-start recommendations use explicit cohort and popularity formulas.
5. **Artifact discipline:** metadata, category maps, metrics, model weights, and recommendation outputs are saved separately.

15 Limitations and Modeling Caveats

A rigorous analysis should also discuss limitations.

1. **Negative sampling is approximate:** sampled negatives may include pairs that are positive in unobserved future data.
2. **Transductive evaluation:** validation and test edges only include users and bundles already present in the training graph.
3. **Dot-product scoring is simple:** it is efficient, but it may be less expressive than a learned pairwise scoring network.

4. **Cold-start is heuristic:** unseen users are handled by cohort popularity rather than by an inductive user-feature encoder connected to the graph.
5. **Popularity bias can persist:** both training data and fallback rules may favor already-popular bundles.

16 Compact Mathematical Summary

The pipeline can be summarized as follows:

$$\begin{aligned}
&\text{Build graph: } G = (\mathcal{U} \cup \mathcal{B}, \mathcal{E}), \\
&\text{Encode features: } h_u^{(0)} = f_U(x_u), \quad h_b^{(0)} = f_B(x_b), \\
&\text{Message pass: } h_v^{(\ell+1)} = \text{GraphSAGE}^{(\ell)}(h_v^{(\ell)}, \{h_w^{(\ell)} : w \in \mathcal{N}(v)\}), \\
&\text{Score link: } s_{ub} = z_u^\top z_b, \\
&\text{Predict probability: } \hat{p}_{ub} = \sigma(s_{ub}), \\
&\text{Recommend: } \text{TopK}(u) = \arg \text{topk}_{b \in \mathcal{B}} \hat{p}_{ub}.
\end{aligned}$$

17 Conclusion

The supplied code implements a production-oriented heterogeneous GraphSAGE recommender for telecom bundles. It begins with careful relational aggregation, turns historical subscriptions into a typed bipartite graph, learns user and bundle embeddings through message passing, and ranks candidate links by dot-product logits.

The most important lesson is that a strong recommender is not only a neural architecture. It is a pipeline: temporal graph construction, stable feature preprocessing, sampled training, principled validation, artifact management, efficient inference, and robust cold-start behavior all work together to produce deployable recommendations.